

Documento de Diseño de Videojuego (GDD)

Vanguard of Darkness

Version 3.0

Integrantes	Benjamín Matías Pavez Vidal / Nicolás Renato Ramírez Berríos
Asignatura	Programación de Videojuegos IEI-061
Docente	Leopoldo Esteban Rodríguez Bravo
Fecha	30 / 06 / 2026
Sede	Iquique — Chile

1. Historial de Revisión

Versión	Autor	Fecha	Cambios
1.00	B. Pavez / N. Ramírez	20/04/2026	Planteamiento inicial: Level 1, un enemigo, animaciones completas, HUD, menú de pausa, música de fondo, parallax continuo con el player.
2.00	B. Pavez / N. Ramírez	18/05/2026	Efectos de sonido y visuales, menú principal, 3 enemigos nuevos, Level 2, pantalla de carga, NPCs, marcador de tiempo, minimapa.
3.00	B. Pavez / N. Ramírez	30/06/2026	Multijugador cooperativo online con Unity Relay completamente implementado. Player 2 como hada (posesión de enemigos + 3 buffos). Sistema de patrullaje autónomo para todos los enemigos. EnemySpawner con spawn en runtime. Level 3 con Boss Demonio. UI de P2. CoopManager/RelayManager/CoopNetworkManager.

2. Introducción y Alcance

2.1. Concepto Principal (High Concept)

Vanguard of Darkness es un juego de plataformas 2D con estética pixel art jugable en modo solitario o en modo cooperativo online para dos jugadores.

En modo solitario, el jugador 1 recorre niveles con enemigos y trata de mejorar su tiempo record. En modo cooperativo, el jugador 2 encarna un hada de apoyo que asiste a P1 con tres tipos de buffos (velocidad, daño y curación) y la capacidad de poseer temporalmente a los enemigos para convertirlos en aliados. La cooperación está implementada con Unity Relay (Unity Gaming Services), permitiendo jugar online sin configuración de router. El Host genera un código de sala de hasta 6 caracteres que el Cliente ingresa para unirse.

2.2. Convenciones de Estilo

Texto normal: características confirmadas e implementadas.

Texto en cursiva: ideas o funcionalidades en discusión.

- PascalCase para clases y MonoBehaviours (PlayerController, EnemyBase).
- camelCase para variables privadas (currentHealth, jumpForce).
- UPPER_SNAKE_CASE para constantes (KEY_VOLUME).

- Escena 0 = Menú Principal. Escenas numeradas por build index.
- Variables en Inspector documentadas con [Header] y [Tooltip].
- PlayerController usa el patrón partial class (3 archivos).

3. Especificaciones del Sistema

3.1. Hardware y Software

- Plataforma: PC (Windows, Mac, Linux).
- Motor: Unity 6 (6000.4.0f1).
- Resolución base: 1920x1080 (16:9). Opción de 1280x720 en ajustes.
- Estética: Pixel Art 2D.
- Red: Unity Gaming Services — Relay + Authentication anónima. NAT traversal sin configuración de router.
- Networking: Netcode for GameObjects (NGO), servidor autoritativo.
- Persistencia: JSON en disco (SaveSystem) + PlayerPrefs (RecordSystem, OptionsManager).

4. Mecánicas de Juego y Objetivos

4.1. Modos de Juego

Modo Solitario

El jugador 1 recorre los niveles en solitario. El objetivo es llegar a la meta en el menor tiempo posible. El timer penaliza (+90 s) la muerte del jugador y premia (-X s) eliminar enemigos. Los records se guardan por nivel. Soporta guardado y carga de partida.

Modo Cooperativo Online (Completamente Implementado)

Dos jugadores conectados vía Unity Relay. El Host crea la sala y obtiene un código; el Cliente lo ingresa. Ambos comparten el mismo timer sincronizado. No soporta guardado de partida (botones ocultos automáticamente en modo multijugador).

- Jugador 1 (P1 / Platformer): personaje principal que avanza, combate y recibe buffos de P2.
- Jugador 2 (P2 / Hada): apoyo kinematic que vuela junto a P1, posee enemigos y aplica buffos.

4.2. Acciones de P1 (Platformer)

- Movimiento horizontal: A (izquierda) / D (derecha).
- Salto / Doble Salto: W, Espacio o Flecha Arriba.
- Wall Slide / Wall Jump: deslizarse por pared y saltar al presionar salto contra superficie vertical.
- Dash: Shift o Clic Derecho. 1 carga, recarga en 1.5 s, velocidad 13 u/s durante 0.3 s.
- Ataque melee: Clic Izquierdo o J. OverlapCircle de rango 0.6. Cooldown configurable.
- Interacción NPC: E.
- Indicadores de buffo: sprite azul (velocidad), naranja (daño), flash verde breve (curación).

4.3. Acciones de P2 (Hada Cooperativa)

- Posesión de enemigos: Clic Derecho sobre enemigo en rango. Sprite de hada se oculta.
 - Mover enemigo poseído: A / D.
 - El enemigo poseído daña a otros enemigos por colisión.
 - El EnemyLongSwordKnight (jefe) no puede ser poseído.
 - Soltar posesión: Clic Derecho. Activa cooldown de 60 s.
 - Si el enemigo poseído muere, la posesión se rompe y aplica cooldown.
- Buffo de Velocidad: Q. Aumenta velocidad de P1. Tiñe a P1 de azul.
- Buffo de Daño: E. Multiplica el daño de P1. Tiñe a P1 de naranja.
- Curación: R. Restaura vida a P1. Flash verde.
- HUD: muestra cooldowns en ingles (Speed Q ready / Damage E Xs / Healing R ready / Possess ready). Descubierta por nombre en runtime.

4.4. Sistema de Timer y Penalizaciones

- Timer cuenta el tiempo transcurrido en el nivel (no es cuenta regresiva).
- Muerte de P1: +90 s en AMBAS maquinas (vía ClientRpc).

- Muerte de enemigo: reducción configurable por enemigo aplicada en OnDieLocal() (todas las maquinas).
- Al terminar el nivel el tiempo se compara con el record y se actualiza si es mejor.

5. Inteligencia Artificial y Comportamiento del Mundo

5.1. Sistema de Patrullaje Autónomo

Todos los enemigos heredan el sistema de patrullaje de EnemyBase. El sistema es autónomo por distancia: el enemigo se desplaza +- patrolDistance unidades desde su posición de spawn, sin waypoints manuales. Esto es fundamental para el EnemySpawner, que instancia prefabs compartidos en runtime y no puede asignar waypoints por instancia desde el Inspector.

Funcionamiento de HandlePatrol():

- Se mueve en una dirección hasta alcanzar spawnPosition +- patrolDistance.
- Espera patrolWaitTime segundos en cada extremo antes de invertir dirección.
- Si CheckPatrolGroundAhead() devuelve false (borde de plataforma), invierte antes de caer.
- Las subclases sobrescriben SetFacing() para actualizar su NetworkVariable de dirección o Flip() local.
- Al detectar al jugador, isChasing = true y la subclase toma el control del movimiento.

5.2. Lógica de Enemigos

EnemyFollow — Slime

Enemigo terrestre básico. Patrulla y persigue al jugador al entrar en su radio de detección. Usa raycast para detectar bordes de plataforma y evitar caídas. Daño por contacto.

EnemyLongSwordKnight — Caballero (Jefe)

Enemigo tipo jefe terrestre. Patrulla, persigue y ejecuta ataque en área con fases de anticipación y recuperación. Detección de bordes opcional. Su muerte desbloquea la meta del nivel. No puede ser poseído por P2.

EnemyFlyingShooter — Ángel Caído (Volador)

Enemigo volador (gravityScale=0). Movimiento sinusoidal en Y. Al detectar al jugador se acerca manteniendo distancia y dispara proyectiles cada fireCooldown segundos. En patrullaje flota a mitad de amplitud. Sin detección de bordes (vuela sobre vacíos).

EnemyProjectile

Proyectil spawnado como NetworkObject. Movimiento lineal hacia la posición del jugador al momento del disparo. Se destruye por tiempo o impacto aplicando daño.

Boss Demonio

Cuarto tipo de enemigo disponible en el EnemySpawner como prefab independiente. Actúa como jefe del nivel final. Su muerte es el requisito para la victoria final del modo cooperativo.

5.3. Interactividad del Entorno

- Trampas estáticas: antorchas y pinchos con tag 'Damage'. Daño por contacto al jugador.
- DeathZone: trigger en bordes del mapa. Aplica 999 de daño a cualquier entidad que caiga fuera.

- NPCs: diálogo con tipeo progresivo. Bloquea controles del jugador mientras dura la conversación.
- LevelGoal: trigger de meta bloqueada hasta cumplir el requisito del boss. Al desbloquearse se vuelve opaca.

6. Arquitectura Técnica y Estructuras de Datos

6.1. Diagrama de Clases

Clase / Script	Responsabilidad
ENTIDADES	
PlayerController	Controlador principal de P1 (partial class). Ciclo de vida NGO, HUD, sincronización de animaciones, RPCs de daño/curación/VFX. Coordina partials de movimiento y combate. Busca la barra de vida en la escena en runtime para actualizarla en ambas maquinas.
PlayerControllerMovement	Partial de P1: movimiento horizontal (5 u/s), sprint/dash (13 u/s, 0.3 s, 1 carga, CD 1.5 s), salto doble, wall-slide/jump, flip de sprite, lectura de inputs. Sincroniza animaciones al servidor vía ServerRpc.
PlayerControllerCombat	Partial de P1: ataque melee (OverlapCircle rango 0.6), knockback, corrutinas de buffo de velocidad y daño con tinte de sprite, flash de curación. Expone ApplySpeedBuff / ApplyDamageBuff / Heal para P2.
Player2Controller	Controlador de P2 (hada cooperativa). Sigue a P1 de forma kinematic (pasa a través de geometría). Posesión de enemigos (clic der., cooldown 60 s). Buffos a P1: velocidad (Q), daño (E), curación (R). HUD descubierto por nombre en runtime. Sincroniza estado de posesión vía NetworkVariable.
PlayerSoundController	Gestiona SFX del jugador (ataque, daño recibido, salto, dash, poder, muerte) vía AudioSource.PlayOneShot.
PlayerEffectsController	VFX del jugador: partículas de polvo al saltar, sprite de tajo al atacar con escala invertida según dirección.
PlayerTargetFinder	Utilidad estática que cachea la referencia al PlayerController (P1) entre frames. Usada por todos los enemigos y por P2Controller para encontrar al objetivo sin FindObjectOfType repetitivo.
DestroyVFX	Autodestruye un GameObject de efecto visual tras un tiempo configurable (lifetime).
ENEMIGOS	
EnemyBase	Clase base de todos los enemigos. NetworkVariable de vida y de posesión. Sistema de patrullaje autónomo por distancia (spawnPosition +- patrolDistance). Knockback, stun, muerte con fade-out y despawn NGO. Virtuals SetFacing() y CheckPatrolGroundAhead() para personalización por subclase. OnDieLocal() corre en todos los clientes y aplica la reducción de tiempo.
EnemyFollow (Slime)	Enemigo terrestre que patrulla y persigue al jugador. Override de CheckPatrolGroundAhead() con raycast para evitar caídas. Aplica daño por contacto. Hereda patrullaje autónomo de EnemyBase.
EnemyLongSwordKnight	Enemigo tipo jefe terrestre. Patrulla, persigue, y ejecuta ataque en área (OverlapCircle) con fases de anticipación y recuperación. Detección de bordes configurable. Requisito para desbloquear la meta de su nivel. NO puede ser poseído por P2.
EnemyFlyingShooter	Enemigo volador (gravityScale=0). Movimiento en curvas en Y. Persecución a distancia con disparo de proyectiles (EnemyProjectile) cada fireCooldown segundos. Patrullaje mantiene el flote sinusoidal a mitad de amplitud. Override de SetFacing() usa Flip() local.
EnemyProjectile	Proyectil de EnemyFlyingShooter spawnado como NetworkObject. Movimiento lineal hacia el jugador, daño por contacto, autodestrucción por tiempo o impacto.

MANAGERS Y SISTEMAS	
GameManager	Singleton central. Timer con ModificarTiempo (penalizaciones/recompensas). Pausa, victoria (WinLevel), guardado-y-salida, navegación de escenas. Oculta opciones de solitario en multijugador. Llama RecordSystem.TrySetRecord al ganar.
CoopManager	Gestión de sesión cooperativa con DontDestroyOnLoad. Activa P2HUD, gestiona transiciones de nivel, comunica victoria/derrota al CLIENTE a través de PlayerController (intermediario NGO). Solo campo visible: P2HUD.
CoopNetworkManager	Ciclo de vida NGO: spawn de jugadores (1.5 s delay), spawn de enemigos (2.5 s delay en relay), panel de espera de P2, manejo de desconexiones y transiciones de escena. Modo Solitario (host local, puerto libre del SO) y modo Relay.
RelayManager	Wrapper de Unity Gaming Services. Inicializa UGS y Authentication (anónimo). Crea sesión como Host con ISession y Relay. Une como Cliente por código de sala. DontDestroyOnLoad.
MainMenuManager	Menú principal: nueva partida, continuar guardado, opciones, scoreboard, crear sala (Host) y unirse por código (Cliente). Mensajes de estado en ingles.
SaveSystem	Singleton de persistencia JSON en disco. Guarda escena, posición, vida, tiempo y enemigos vivos. DontDestroyOnLoad.
RecordSystem	Singleton de records por nivel en PlayerPrefs (JSON). Expone TrySetRecord y GetAll para el Scoreboard del menú.
LoadingScreenManager	Carga asíncrona de escenas con barra de progreso suavizada (fake progress).
OptionsManager	Volumen y resolución con persistencia en PlayerPrefs.
NetworkModeData	Clase estática con enum Mode {Solitario, Host, Cliente}. Persiste la selección de modo entre escenas sin MonoBehaviour.
SISTEMAS DE NIVEL Y ENTORNO	
EnemySpawner	Spawnea enemigos como NetworkObjects en puntos configurados. Tres estrategias: SpawnPointConfig, tags, ciclo por defecto. Delay 2.5 s en relay para garantizar que clientes estén listos.
SpawnPointConfig	Componente en spawn points que declara el tipo de enemigo (Slimes, Enemyfly, Knight, BossDemon). Consultado por EnemySpawner y LevelGoal.
LevelGoal	Meta del nivel. Puede requerir la muerte de un boss (por nombre y radio de zona). Solitario: WinLevel(). Cooperativo: CoopManager.OnGoalReached(). Soporte para victoria final vs. transición de nivel.
Dialogue	Sistema de diálogo NPC con tipeo progresivo. Bloquea controles del jugador (IsTalking) durante la conversación.
CameraFollow	Cámara suave con Vector3.SmoothDamp. SnapToTarget() para teleport instantáneo usado por P2 al spawnear.
DeathZone	Trigger de muerte instantánea (999 daño) para jugadores y enemigos fuera del mapa.
Parallax / ColumnParallax	Desplazamiento UV y de posición según movimiento de cámara para efecto de profundidad. Se pausa con el juego.
SaveData / LevelRecord	Modelos serializables. SaveData: escena, posición, vida, tiempo, enemigos vivos. LevelRecord: sceneIndex, levelName, bestTime.

6.2. Arquitectura de Red

El juego usa Netcode for GameObjects (NGO) con modelo servidor-autoritativo:

- Toda la lógica de IA corre exclusivamente en el servidor (if (!IsServer) return).
- Posiciones de enemigos sincronizadas vía NetworkTransform.
- Vida de jugadores y enemigos usa NetworkVariable<int> (escritura solo en servidor).
- Efectos visuales y sonido se propagan vía ClientRpc (patrón: ClientRpc llama método virtual protegido, override en subclases).
- P2Controller usa NetworkVariable<bool> (netIsPossessing) para ocultar sprite en todas las maquinas.
- CoopManager comunica a través de PlayerController como intermediario NGO (CoopManager no es NetworkObject por DontDestroyOnLoad).

Flujo de conexión cooperativa:

- Host: MainMenuManager -> RelayManager.CreateRelayHost() -> obtiene código -> carga Level 1.
- Cliente: ingresa código -> RelayManager.JoinRelayAsClient() -> carga Level 1 vía NGO.
- En cada escena: CoopNetworkManager spawna P1 a 1.5 s y enemigos a 2.5 s.

6.3. Maquinas de Estado (FSM)

Jugador 1 — PlayerController

Estado	Condición de Entrada	Descripción / Comportamiento
Idle	horizontalInput==0 y isGrounded	Quieto en suelo. Animator: movement=0, IsGrounded=true.
Running	horizontalInput!=0 y isGrounded	Movimiento a walkSpeed (5 u/s). Puede iniciar Dash.
Jumping	Space/W/Arriba, jumpsRemaining>0	Impulso vertical. Hold prolongado aplica jumpHoldForce hasta 0.2 s.
Falling	!isGrounded && yVelocity<-0.1	Caída libre. IsFalling=true en Animator.
WallSliding	isTouchingWall && !isGrounded && input hacia pared	Limita caída a wallSlideSpeed (3 u/s). Permite WallJump.
Dashing	Shift / Clic Der., charges>0	Impulso a 13 u/s durante 0.3 s. 1 carga max., recarga en 1.5 s.
Attacking	Clic Izq. / J, time>=nextAttackTime	OverlapCircle (rango 0.6) daña enemigos en rango. Cooldown configurable.
KnockedBack	TakeDamage() invocado	Knockback + trigger Hurt. Sin control durante 0.2 s.
Dead	networkHealth<=0	isDead=true. Velocidad=0. +90 s al timer (ambos clientes). Respawn en 1.5 s.
Talking	IsTalking==true (NPC activo)	Bloquea movimiento y combate. Cámara y animaciones activas.

Jugador 2 — Player2Controller (Hada)

Estado	Condición de Entrada	Descripción / Comportamiento
Following	Siempre activo (servidor)	Hada kinematic sigue a P1 con MoveTowards a offset (0.8, 0.5). Velocidad mínima 8 u/s.
Possessing	Clic Der. en enemigo, dentro del radio	Sprite oculto (netIsPossessing=true). Mueve enemigo con A/D vía ServerRpc.
Cooldown Posesión	Soltar posesión o enemigo muere	60 s sin poseer. Countdown visible en HUD.
Buffing	Q / E / R presionado, cooldown==0	Q: Speed buff (P1 azul). E: Damage buff (P1 naranja). R: Curación + flash verde.

EnemyFollow — Slime

Estado	Condición de Entrada	Descripción / Comportamiento
Patrol	distanceToPlayer >= detectionRadius	Patrullaje autónomo +- patrolDistance. Espera en extremos. Detección de bordes con raycast.
Chase	distanceToPlayer < detectionRadius	Se mueve hacia P1 verificando suelo adelante.
Stop	distanceToPlayer <= stoppingDistance	Detiene movimiento. Daño por contacto activo.
Stunned	TakeDamage()	isStunned durante stunDuration. Knockback. Sin movimiento.
Dead	networkHealth<=0	Collider off. Fade-out. Reduce tiempo global (OnDieLocal en todos los clientes). Despawn NGO.

EnemyLongSwordKnight — Caballero

Estado	Condición de Entrada	Descripción / Comportamiento
Patrol	distanceToPlayer >= detectionRadius	Patrullaje autónomo. Detección de bordes configurable.
Chase	distanceToPlayer < detectionRadius, fuera de attackRange	Movimiento a speed hacia P1.
Attack (Prep)	OverlapCircle detecta P1 en attackRange	Detiene movimiento. Trigger 'Attack'. Espera attackAnticipationTime.
Attack (Hit)	Tras anticipación	OverlapCircle aplica swordDamage. Espera attackRecoveryTime.
Stunned	TakeDamage()	isStunned. Limpia triggers de anim. Knockback.
Dead	networkHealth<=0	Collider off. Fade-out. Desbloquea LevelGoal del nivel.

EnemyFlyingShooter — Ángel Caído

Estado	Condición de Entrada	Descripción / Comportamiento
Patrol (Flota)	distanceToPlayer >= detectionRange	Patrullaje +- patrolDistance + flote sinusoidal en Y al 50% de amplitud.
Chase	distanceToPlayer < detectionRange	Persecución a posición objetivo (P1 - followDistance). Flote sinusoidal completo.
Shooting	fireTimer <= 0	Instancia EnemyProjectile orientado a P1. Trigger 'Attack'. Reset fireTimer.
Stunned	TakeDamage()	linearVelocity=0. Flash rojo (FlashRoutine). isStunned durante stunDuration.
Dead	networkHealth<=0	gravityScale=1 (caída física). Collider off. Fade-out. Despawn NGO.

7. Estructura de Niveles

7.1. Niveles Implementados

Escena	Nombre	Enemigos	Condición de Meta
0	Menú Principal	—	—
1	Level 1	Slimes, Fallen Angels	Llegar a la meta (sin boss requerido)
2	Level 2	Slimes, Fallen Angels, LongSwordKnight	Eliminar al EnemyLongSwordKnight de la zona
3	Level 3	Todos los tipos + Boss Demonio	Eliminar al Boss Demonio (victoria final del modo coop)

7.2. EnemySpawner — Spawn en Runtime

Los enemigos se instancian en runtime por el EnemySpawner como NetworkObjects. El prefab a spawnear en cada punto se determina con tres estrategias en orden de prioridad:

- SpawnPointConfig: componente en el spawn point que declara el tipo (Slimes, Enemyfly, Knight, BossDemon).
- Tags de GameObject: SpawnSlimes, SpawnEnemyfly, SpawnKnight, SpawnBossDemon.
- Ciclo por defecto: índice % 3 entre los tres tipos principales.

Delay de spawn: en modo relay el EnemySpawner espera 2.5 s (WaitForSecondsRealtime) para asegurar que P1, P2 y el cliente NGO estén listos antes de recibir spawn messages.

7.3. Editor de Niveles — Herramientas Unity

- Tilemap Suelo/Plataformas: colisión con Tilemap Collider 2D + Composite Collider 2D.
- Tilemap Decoración: capa visual sin colisión.
- Tilemap Peligros: tiles con tag 'Damage' (antorchas, pinchos).
- Tile Palette: edición por Rect (áreas grandes) y Single (detalles finos).
- Rule Tile: selección automática de sprite según vecinos (esquinas, bordes, relleno).
- Sprite Atlas: agrupación de tiles por tema para reducir draw calls.

Flujo de construcción: boceto -> capa de colisión -> decoración -> Prefabs de enemigos (EnemySpawner) -> LevelGoal -> DeathZone en bordes.

8. Sistemas de Persistencia y Progression

8.1. SaveSystem — Guardado de Partida (Solo Solitario)

Serializa SaveData a JSON en Application.persistentDataPath. Datos guardados:

- Índice de escena activa.
- Posición del jugador (X, Y).
- Vida actual.
- Tiempo transcurrido.
- Lista de nombres de enemigos vivos.

8.2. RecordSystem — Records y Scoreboard

Almacena el mejor tiempo por nivel en PlayerPrefs como JSON (AllRecords -> lista de LevelRecord). TrySetRecord actualiza el record si el nuevo tiempo es mejor. GetAll() alimenta el Scoreboard del menú principal.

8.3. OptionsManager — Configuración

Volumen global (AudioListener.volume) y resolución de pantalla. Ambos persisten en PlayerPrefs y se recuperan al iniciar.

9. Decisiones de Diseño e Implementación

9.1. Partial Class Pattern (PlayerController)

PlayerController se divide en tres archivos partial: PlayerController.cs (ciclo de vida NGO, HUD, RPCs), PlayerControllerMovement.cs (física y controles) y PlayerControllerCombat.cs (combate, knockback, buffos). Cada archivo mantiene una responsabilidad clara y un tamaño manejable.

9.2. Patrullaje por Distancia vs. Waypoints

Se eligió patrullaje autónomo por distancia (spawnPosition +- patrolDistance) porque los enemigos se instancian desde prefabs compartidos vía EnemySpawner. No es posible asignar waypoints individuales en el Inspector de un prefab de red que sera instanciado múltiples veces con distinta posición.

9.3. Comunicación CoopManager -> Cliente

CoopManager usa DontDestroyOnLoad, lo que lo saca de la escena NGO. Sus [ClientRpc] no llegan al cliente. Solución: enviar mensajes a través de PlayerController (spawneado como NetworkObject). Métodos: MostrarVictoriaClientRpc, MostrarDerrotaClientRpc, CargarSiguienteNivelClientRpc, NotificarTransicionClientRpc.

9.4. Timer Sincronizado Sin NetworkVariable

GameManager existe en ambas maquinas de forma independiente. El timer se sincroniza llamando GameManager.instance.ModificarTiempo() localmente en cada cliente cuando ocurren eventos globales: muerte de P1 (vía DieClientRpc) y muerte de enemigo (vía DieEffectsClientRpc -> OnDieLocal).

9.5. HUD de P2 Descubierta en Runtime

Player2Controller es un prefab de red spawneado en runtime; sus campos [SerializeField] no pueden asignarse en el Inspector de la escena. Solución: GameObject.Find() por nombre en OnNetworkSpawn() (BuscarHUDP2EnEscena), el mismo patrón ya usado por PlayerController para la barra de vida.

9.6. Delay de Spawn en Modo Relay

En modo relay, los jugadores se spawnean a 1.5 s y los enemigos a 2.5 s (WaitForSecondsRealtime, ignora timeScale). Este delay garantiza que el cliente NGO haya inicializado su contexto en la nueva escena y que los spawn messages no lleguen con deferred timeout. En modo solitario no hay delay porque el servidor es local.

9.7. Eliminación de mecánicas

Se elimino la mecánica de agacharse y el mini mapa, esto se debe a que como equipo consideramos que el agacharse no tenía uso en el juego y el mini mapa rompía no solo la estética del juego sino que también chocaba con la idea que teníamos de este.